

CÂMERA E PERMISSÕES

Desenvolvimento do App TechCall: Configuração de Ambiente e Captura de Imagens

11 de agosto de 2026

1. VISÃO GERAL DA AULA

Esta aula marca a transição do ambiente online (Expo Snack) para o desenvolvimento profissional local. Iniciaremos o projeto **TechCall**, um aplicativo para técnicos de suporte em campo. O foco hoje é configurar o ambiente e permitir que o técnico anexe fotos de equipamentos aos chamados.

Ao final desta aula, o aluno será capaz de:

1. Criar um projeto Expo localmente no computador.
2. Executar o aplicativo no próprio celular via **Expo Go**.
3. Criar uma tela de formulário funcional.
4. Solicitar e tratar permissões nativas de câmera e galeria.
5. Capturar, exibir e remover fotos no aplicativo.
6. Tratar erros de cancelamento e permissão negada sem causar o fechamento do app (*crash*).

Materiais Necessários

- Computador (Windows ou macOS) com VS Code instalado.
- Celular com o aplicativo **Expo Go** instalado (disponível na Play Store ou App Store).

2. PREPARAÇÃO

É fundamental garantir que o **Node.js** esteja instalado. O Node é o "motor" que permite rodar JavaScript no computador.

1. **Verifique:** Abra o terminal (PowerShell no Windows ou Terminal no macOS) e digite `node -v`. Se aparecer algo como `v22.x.x`, está pronto.
 2. **Instale:** Se não for reconhecido, baixe a versão **LTS** (mais estável) em nodejs.org (<https://nodejs.org>).
 3. **npm:** Ao instalar o Node, você ganha o **npm** (gerenciador de pacotes). Verifique com `npm -v`. Ele será usado para instalar as bibliotecas do nosso app.
-

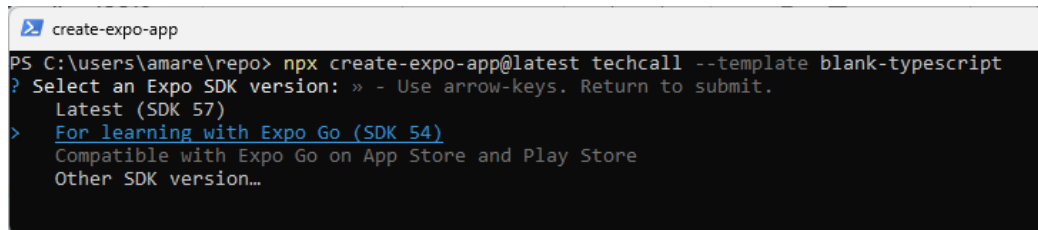
3. CRIANDO O PROJETO EXPO

O **Expo** é um conjunto de ferramentas que facilita a criação de apps. Diferente do Snack, agora o código vive no seu computador.

1. **Navegar:** No terminal, vá para a pasta onde o projeto será criado (ex: c:\trabalho).
2. **Criar o projeto:** Use o comando **npx**:

```
npx create-expo-app@latest techcall --template blank-typescript
```

Importante: quando perguntado, escolher a opção SDK 54.



3. **Entrar na pasta:** cd techcall
4. **Abrir no VS Code**

Estrutura de Pastas:

- **node_modules:** Onde ficam as bibliotecas (não mexer).
- **App.tsx:** O coração do app, onde tudo começa.
- **package.json:** A "lista de compras" do projeto (mostra o que está instalado).

4. PRIMEIRA EXECUÇÃO NO CELULAR (EXPO GO)

O **Expo Go** permite que seu celular "assista" ao código que você escreve no computador em tempo real.

1. Instale a dependência expo/ngrok:

```
npx expo install @expo/ngrok
```

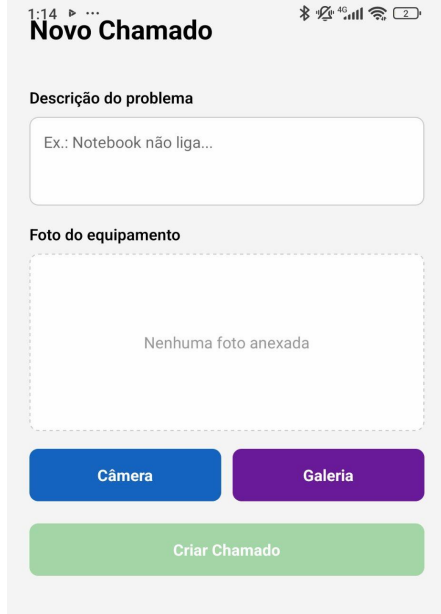
Importante: Esse passo só é necessário só na primeira vez.

2. **Iniciar o servidor:**

```
npx expo start --tunnel
```

3. Aguarde aparecer um endereço público (algo como **exp://xxxxx.exp.direct:80**).
 4. **QR Code:** Um código aparecerá no terminal.
 5. **Conectar:** Abra o Expo Go no celular e escaneie o código.
-

6. INSTALANDO A BIBLIOTECA DE IMAGEM



Para acessar a câmera, precisamos de um "módulo" extra. No terminal (pode abrir um novo ou parar o atual com Ctrl+C), digite:

```
npx expo install expo-image-picker
```

Observe que a dependência **expo-image-picker** foi acrescentada no arquivo **package.json**.

6. CÓDIGO GUIADO (APP TECHCALL)

Vamos transformar o app em uma ferramenta de trabalho. O técnico precisa descrever o problema e tirar uma foto.

Etapa 1: Organização

Crie uma pasta chamada **src** na raiz do projeto. Dentro dela, crie a pasta **screens**. Lá, crie o arquivo **NewCallScreen.tsx**.

Etapa 2: Configurar o App.tsx

Substitua todo o conteúdo do arquivo App.tsx original por este:

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, View } from 'react-native';
import NewCallScreen from '../src/screens/NewCallScreen';

export default function App() {
  return (
    <View style={styles.container}>
      <StatusBar style="auto" />
      <NewCallScreen />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
  },
});
```

Etapa 3: Implementar a Tela de Novo Chamado

No arquivo `src/screens/NewCallScreen.tsx`, insira o código abaixo. Leia os comentários para entender a lógica:

```
import React, { useState } from 'react';
import {
  Alert,
  Image,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
} from 'react-native';
import * as ImagePicker from 'expo-image-picker';

export default function NewCallScreen() {
  // Estados: "caixinhas" que guardam o que o usuário digita ou a foto que
  // escolhe
  const [description, setDescription] = useState('');
  const [photoUri, setPhotoUri] = useState<string | null>(null);

  // Função para tirar foto com a câmera
  async function handleTakePhoto() {
    // 1. Pede permissão para usar a câmera
    const permission = await ImagePicker.requestCameraPermissionsAsync();

    if (!permission.granted) {
      Alert.alert('Permissão negada', 'Precisamos da câmera para o
chamado.');
```

```
      return;
    }

    // 2. Abre a câmera
    const result = await ImagePicker.launchCameraAsync({
      allowsEditing: true, // Permite cortar a foto
      aspect: [4, 3],      // Proporção da imagem
      quality: 0.7,        // Qualidade (70% para não pesar o app)
    });

    // 3. Se o usuário não cancelou, salva o endereço (URI) da foto
    if (!result.canceled) {
      setPhotoUri(result.assets[0].uri);
    }
  }

  // Função para escolher da galeria
  async function handlePickFromGallery() {
    const permission = await
ImagePicker.requestMediaLibraryPermissionsAsync();

    if (!permission.granted) {
      Alert.alert('Permissão negada', 'Precisamos acessar suas fotos.');
```

```
    return;
  }

  const result = await ImagePicker.launchImageLibraryAsync({
    allowsEditing: true,
    aspect: [4, 3],
    quality: 0.7,
```

```

});

if (!result.canceled) {
    setPhotoUri(result.assets[0].uri);
}
}

return (
    <ScrollView style={styles.screen} contentContainerStyle={styles.content}>
        <Text style={styles.title}>Novo Chamado</Text>
        <Text style={styles.label}>Descrição do problema</Text>
        <TextInput
            style={styles.input}
            placeholder="Ex.: Notebook não liga..."
            value={description}
            onChangeText={setDescription}
            multiline
        />
        <Text style={styles.label}>Foto do equipamento</Text>
        {/*Renderização Condicional: Se tem foto, mostra a imagem. Se não,
mostra o aviso.*/*}
        { photoUri ? (
            <View>
                <Image source={{ uri: photoUri }} style={styles.photo} />
                <TouchableOpacity
                    onPress={() => setPhotoUri(null)}
                    style={styles.removeButton}>
                    <Text style={styles.removeButtonText}>Remover foto</Text>
                </TouchableOpacity>
            </View>
        ) : (
            <View style={styles.placeholder}>
                <Text style={styles.placeholderText}>Nenhuma foto anexada</Text>
            </View>
        )}

        <View style={styles.buttonRow}>
            <TouchableOpacity style={[styles.button, styles.cameraButton]}
onPress={handleTakePhoto}>
                <Text style={styles.buttonText}>Câmera</Text>
            </TouchableOpacity>
            <TouchableOpacity style={[styles.button, styles.galleryButton]}
onPress={handlePickFromGallery}>
                <Text style={styles.buttonText}>Galeria</Text>
            </TouchableOpacity>
        </View>
        <TouchableOpacity
            style={[styles.button, styles.confirmButton, !description &&
styles.disabledButton]}
            disabled={!description}
            onPress={() => Alert.alert('Sucesso', 'Chamado registrado
localmente!')}
        >
            <Text style={styles.buttonText}>Criar Chamado</Text>
        </TouchableOpacity>
    </ScrollView>
);
}

const styles = StyleSheet.create({
    screen: { flex: 1, backgroundColor: '#f5f5f5' },

```

```
content: { padding: 20, paddingBottom: 40 },
title: { fontSize: 24, fontWeight: 'bold', marginBottom: 20 },
label: { fontSize: 14, fontWeight: '600', marginTop: 16, marginBottom: 8 },
input: { borderWidth: 1, borderColor: '#ccc', borderRadius: 8,
backgroundColor: '#fff', padding: 12, minHeight: 80, textAlignVertical: 'top'
},
placeholder: { height: 160, borderRadius: 8, borderWidth: 1, borderColor:
'#ccc', borderStyle: 'dashed', backgroundColor: '#fff', alignItems: 'center',
justifyContent: 'center' },
placeholderText: { color: '#999' },
photo: { width: '100%', height: 200, borderRadius: 8 },
removeButton: { marginTop: 8, alignItems: 'center' },
removeButtonText: { color: '#d32f2f', fontWeight: 'bold' },
buttonRow: { flexDirection: 'row', marginTop: 16, gap: 10 },
button: { flex: 1, borderRadius: 8, padding: 14, alignItems: 'center' },
cameraButton: { backgroundColor: '#1565c0' },
galleryButton: { backgroundColor: '#6ab9a' },
confirmButton: { backgroundColor: '#2e7d32', marginTop: 20 },
disabledButton: { backgroundColor: '#a5d6a7' },
buttonText: { color: '#fff', fontWeight: 'bold' },
});
```

7. EXPLICAÇÃO DOS CONCEITOS

6. **useState:** É uma variável que, quando alterada, avisa ao React para desenhar a tela de novo. Usamos para o texto da descrição e para o endereço da foto.
7. **Async/Await:** Operações como abrir a câmera levam tempo. O await diz ao app: "espere o usuário terminar de tirar a foto antes de continuar para a próxima linha".
8. **Permissões:** O celular é um ambiente seguro. O app deve pedir licença (request...PermissionsAsync) antes de acessar dados privados.
9. **URI:** É o "caminho" onde a foto ficou guardada temporariamente no celular.
10. **result.canceled:** Sempre verificamos se o usuário desistiu da foto para evitar que o app tente exibir algo que não existe e trave.

8. ERROS COMUNS E SOLUÇÕES

Sintoma	Causa Provável	Solução
"node" não é reconhecido	Node não instalado ou terminal antigo	Instale o Node LTS e REABRA o terminal.
QR Code não conecta	Redes Wi-Fi diferentes	Conecte PC e Celular no mesmo Wi-Fi ou -tunnel.
App fecha ao tirar foto	Falta verificar result.canceled	Adicione o if (!result.canceled) no código.
Erro de permissão	Permissão negada no sistema	Vá nas configs do celular > Expo Go > Permissões.